

2. Hotel / Flight Booking System

Interviewer: Design a hotel booking system — think Booking.com or Expedia. Users search for a city, pick dates and guest count, browse a list of hotels, and book a room. Let's start there.

Candidate: Good one — this problem has two very different personalities living under one roof: a **read-heavy search funnel** (thousands of people browsing) feeding a **tiny, correctness-critical write path** (a handful of people actually booking). Let me clarify before I commit to a shape.

1. Clarifying the requirements

Candidate: A few questions: is this hotels only, or should I also cover flights (the core inventory problem feels the same)? Is availability per hotel, or does it vary **night by night** within a stay (Friday sold out, Saturday open)? Is there a checkout flow — reserve, then pay — or does browsing go straight to "confirmed" in one shot? And what's the one thing that must never happen — I assume **selling the same room for the same night to two different guests**?

Interviewer: Focus on hotels; flights are a bonus at the end. Yes — availability is per calendar night, a 3-night stay can be full on night 2 and open on nights 1 and 3. There's a short "reserve → pay" window, not instant confirm. And yes: **never oversell a room-night**. That's the one invariant you may not break, ever, under any load.

Candidate: Then let me write it down.

Functional - `GET /search?city&checkin&checkout&guests` → ranked list of hotels/room types with a starting price and "X rooms left." - `GET /hotels/{id}` → detail page. `POST /bookings` → place a **hold** on a room for a date range. `POST /bookings/{id}/pay` → charge and confirm. `POST /bookings/{id}/cancel` → release the room-nights.

Non-functional — and this is the crux of the problem - **Correctness on the write path is non-negotiable.** Two people can never end up holding a confirmed booking for the same room on the same night. - **The read path is a scale problem, not a correctness problem.** Millions of browsers, slightly stale data is fine — nobody's oversold by *looking* at a search result. - **Low search latency** (sub-300ms) so results feel interactive. - **Payment safety:** no double charge, no charge-without-booking.

Candidate: So I want to say this out loud early, because it drives every decision below: **search is a scale problem, solved with replicas/caching/indexes; booking is a correctness problem, solved with transactions and locking.** They deserve different technology.

2. Estimation

Candidate: Let me size both funnels, because they diverge hard.

Inventory:

50,000 hotels x 100 rooms avg x rolling 365-day booking window
= 50,000 * 100 * 365 ~ 1.8 BILLION room-night rows
(this fits a partitioned SQL table fine; it's write-light per row)

Traffic (50:1 browse:book ratio is typical for travel):

10M searches/day → 10M/86,400 ~ 116/s avg, ~1,000-2,000/s at peak
200k bookings/day → 200k/86,400 ~ 2.3/s avg, ~50-100/s at peak

=> Search: ~1,000x the QPS of booking, but tolerant of staleness.

=> Booking: tiny QPS, but a SINGLE wrong answer is a real-world double-booking, an angry guest at a hotel counter, and a refund. Zero tolerance for the race.

This asymmetry – huge read fan-out vs. tiny, strict write – is the whole game.

3. V1 — the simple design

Candidate: Let me sketch a V1 that's obviously correct, then scale it.

```
[ Client ] --GET /search--> [ App Server ] --SQL--> [ Postgres: Hotel/Room/Inventory ]  
[ Client ] --POST /bookings--> [ App Server ] --SQL txn--> [ Postgres ]
```

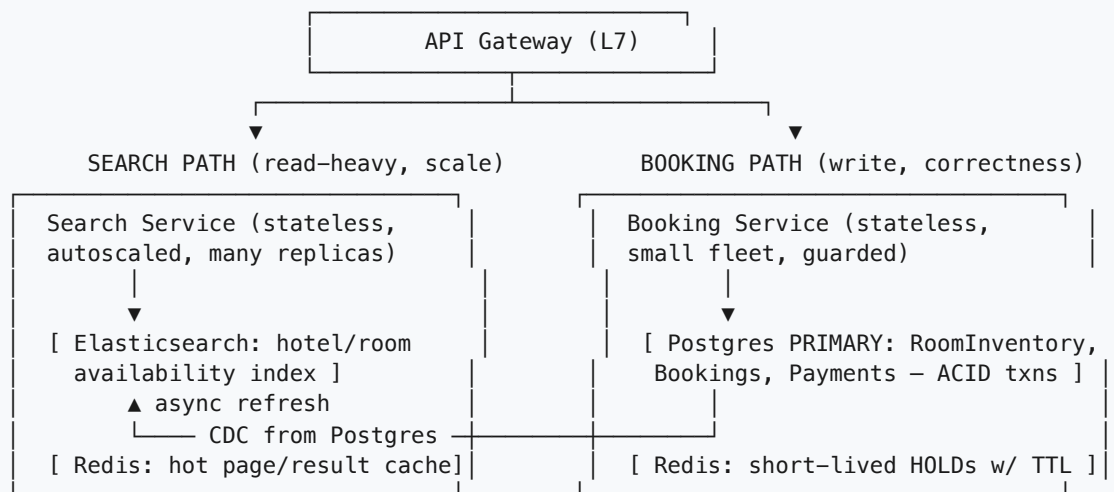
One relational database holds hotels, room types, a **per-date inventory table**, and bookings. Search runs a SQL query with date-range filters; booking runs a transaction that checks and decrements inventory. It's correct — read-for-decision and write-for-commit both happen inside one ACID transaction on one system.

Why this dies at scale: every search query — 1,000-2,000/s at peak — competes with booking transactions for the *same* primary's CPU and I/O. Search doesn't need transactional consistency; forcing it through the transactional primary anyway means a search spike (a promo, a holiday weekend) can slow down or lock out the booking path that actually needs correctness. **Mixing the scale problem and the correctness problem onto one system is the mistake.** I'm going to split them.

4. The split: search side vs. booking side

Interviewer: Split them, then. Walk me through both halves.

Candidate:



Search reads from Elasticsearch + Redis cache — never the transactional primary. Elasticsearch is fed asynchronously (CDC/change-data-capture, or an outbox + Kafka) from Postgres, so it's a denormalized, eventually-consistent *view* of inventory, built for fast filtered/sorted queries (city, date range, price, star rating, guest count). **Booking always reads and writes the Postgres primary, inside a transaction** — it never trusts the search index's number. This is the load-bearing decision in the whole problem: **the search index can be stale by seconds; the booking decision can never be stale by a microsecond.**

5. The data model — room-nights, not a single counter

Interviewer: You mentioned availability is per-night. How's that modeled?

Candidate: The naive model — one `rooms_left` counter per hotel — breaks the moment a stay spans multiple nights with different occupancy. I model inventory **per (hotel, room type, date)** — a "room-night" row:

Hotel

hotel_id (PK), name, city, geo, region_home <- for multi-region, see §9

RoomType (a class of room within a hotel)

room_type_id (PK), hotel_id (FK), name, capacity, base_price

RoomInventory <-- THE CENTRAL, CONTENTED TABLE

hotel_id, room_type_id, date COMPOSITE PK (one row per night)
total_rooms INT e.g. 20 rooms of this type exist
booked_rooms INT currently held or confirmed
version INT optimistic-lock counter

available(date) = total_rooms - booked_rooms

Booking

booking_id (PK), user_id, hotel_id, room_type_id,
checkin_date, checkout_date,
status: HELD | CONFIRMED | CANCELLED | EXPIRED
hold_expires_at, idempotency_key (UNIQUE), created_at

Payment

payment_id (PK), booking_id (FK), amount,
status: PENDING | SUCCEEDED | FAILED,
idempotency_key (UNIQUE), gateway_ref

A 3-night stay touches **3 rows**, one per date, all checked and updated inside **one transaction** — every date must have availability, or the whole booking fails and rolls back. This also makes contention *local*: a hot hotel on a hot Saturday only contends on that hotel's rows for that date, never with unrelated hotels or unrelated nights.

6. Preventing double-booking — the centerpiece

Interviewer: Two guests click "book" on the very last room, same instant. Walk me through exactly what stops both from succeeding.

Candidate: This is a textbook **read-then-write race**:

```
Guest A: SELECT available -> 1      Guest B: SELECT available -> 1
Guest A: available > 0, proceed      Guest B: available > 0, proceed
Guest A: UPDATE booked_rooms += 1    Guest B: UPDATE booked_rooms += 1
-> booked_rooms now EXCEEDS total_rooms. Both "succeeded." Room sold twice.
```

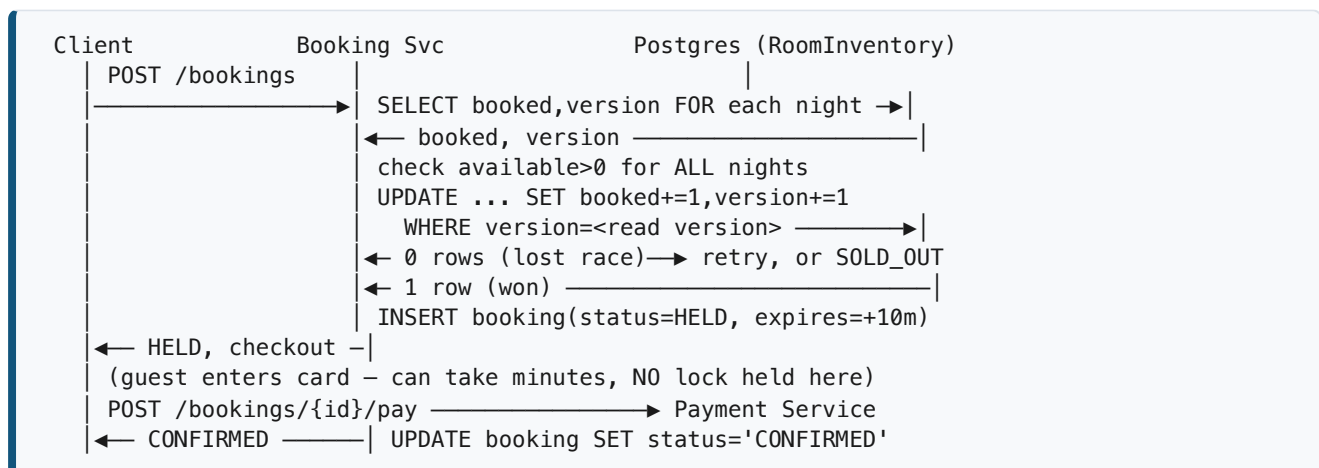
Both read stale data *before* either writes. Two ways to close the gap:

- **Pessimistic — SELECT ... FOR UPDATE** : lock exactly the touched date rows inside the transaction, check `available = total - booked > 0` for every night, update, commit. A concurrent `FOR UPDATE` on the same rows **blocks** until this commits, then re-reads the now-

decremented count and correctly fails. Simple to reason about; the lock is held only for the short transaction (milliseconds), not across a human paying with a credit card.

- **Optimistic — a version column, no blocking:** read `booked_rooms, version`; check `available > 0` in app code; `UPDATE ... SET booked_rooms=booked_rooms+1, version=version+1 WHERE version=<version just read>`. Zero rows updated means someone else won the race in between — retry the read or fail. No lock held at all, just a compare-and-swap.

My recommendation: optimistic locking to win the HOLD, immediately followed by a short-lived hold with a TTL — because the real danger isn't the instant of *deciding*, it's what happens *between* deciding and the guest actually entering a credit card:



Why not just hold the pessimistic row lock across the whole "enter card details" step? Because that lock would sit open for **seconds to minutes** of human typing time on the *hottest, most contended* inventory rows — exactly where it hurts most. The version-check wins the HOLD fast (milliseconds, no lock held across payment); the HOLD itself (with a TTL) protects the room during the slow human/payment part; a background sweeper flips expired unpaid HOLDS back to available.

7. Now scale it — 10M searches/day, then a spike

Interviewer: Now assume a holiday weekend: search traffic 10x's overnight. Walk the read path first, then tell me if the write path even notices.

Candidate: The read path scales exactly like any high-fan-out read system: **more Search Service replicas** (stateless, autoscale on QPS, no shared state), **Elasticsearch shards** by geography/city so no single shard serves the whole world's queries, a **Redis cache** in front of the hottest queries (top cities, this weekend's dates) and detail pages, and a **CDN** for static assets (photos, page shell). The write path **does not notice**, and that's the point of the split: booking QPS tracks how many people actually *commit* to buying, which barely moves even when browse traffic spikes 10x. Because search reads from Elasticsearch/Redis and never touches the primary, a search spike can't starve booking transactions of DB capacity.

Interviewer: What if the *booking* path itself spikes — a single hotel goes viral?

Candidate: That's a **hot-row problem**, not a scale-out-more-replicas problem, because correctness requires all contenders to serialize through the *same* rows. Mitigations: keep the transaction **as short as possible** (just the touched-night rows, commit fast); **shard by `hotel_id`** so unrelated hotels never contend on the same DB shard/partition; and accept that this one hotel's write throughput is capped by row-lock throughput — fine, since even a "viral" hotel is a few hundred rooms, not millions of writers hammering one row (contrast a flash-sale giveaway, a different beast entirely).

8. Async — what leaves the critical path

Interviewer: What happens after `CONFIRMED` — does the guest wait for all of it?

Candidate: No. The guest's confirmation click only needs: **hold validated + payment succeeded + booking row flipped to `CONFIRMED`**. Everything else is async, off a durable event log:

```
[Booking Service] -- payment success, booking CONFIRMED --
| emit "booking-confirmed" event
|
v
[ Kafka: booking-events ] (partitioned by booking_id/hotel_id, durable, replayable)
|
| -> [ Confirmation email/SMS worker ]
| -> [ Elasticsearch indexer ] -- refresh this hotel's availability in the index
| -> [ Analytics / data warehouse ]
| -> [ Hotel's PMS / partner notification ]
```

This is also how the search index stays "close enough" to fresh without being on booking's critical path: every confirmed booking (or cancellation/expiry) emits an event, a consumer updates Elasticsearch a moment later — search sees it in roughly seconds; the guest who just booked never waited for it. **Why Kafka, not direct synchronous calls to each downstream:** a slow email provider or flaky partner webhook must never slow down or fail a booking that already succeeded — Kafka decouples "the booking is real" (committed in Postgres) from "everyone who cares has heard about it" (drains async, replayable if a consumer crashes).

9. Multi-region — data locality and consistency

Interviewer: This is a global product — hotels in Paris, guests in Tokyo. How do you place the data?

Candidate: Two very different things need different placement:

Each hotel has a HOME REGION (a Paris hotel's authoritative RoomInventory + Bookings rows live in an EU Postgres primary). Booking WRITES for that hotel always go there, no matter where the guest sits:

Guest in Tokyo —(booking write)→ routed to EU primary (hotel's home)
→ a few hundred ms extra latency,
ACCEPTABLE: booking is RARE
(2.3/s avg), correctness > speed.

SEARCH reads should be LOCAL and fast everywhere instead: a globally-replicated Elasticsearch index, fed async via CDC from every region's primary, so a Tokyo user's search for Paris hotels hits a nearby replica (same principle as caching, applied on the geography axis) – vs. fanning the query out to every region and merging, which I'd avoid; one global index is simpler and search already tolerates staleness (§4).

The real question: is inventory consistent *across* regions? **Each room-night's authoritative count lives in exactly one region — no cross-region write, no cross-region distributed transaction.** A Paris hotel is never "partially booked" in two regions at once, because every write for it funnels through one primary. Only the **search index copy** is eventually consistent across regions; a guest who commits to book is always re-validated against that one true primary regardless of where they're sitting.

10. Caching — what's safe, what's forbidden

Interviewer: Where exactly does caching go wrong for a problem like this?

Candidate: The rule: **cache what's safe to be stale; never cache the number a booking decision is made from.**

SAFE to cache (cache-aside, TTL):

- Hotel details, photos, reviews (rarely changes, long TTL)
- Search RESULT pages ("rooms from \$180") (short TTL, or served straight from the async-refreshed Elasticsearch index)

NEVER cached – always read from the Postgres PRIMARY, inside the txn:

- "Is there a room-night left for THIS booking, right now"
This is the one number in the entire system that decides real money and a real double-booking. It gets read fresh, checked, and updated atomically (§6) – every single time, no exceptions.

Showing a slightly-stale "3 rooms left" badge in search results is a minor UX blemish worth accepting for speed. *Deciding* to sell a room based on a stale number is a double-booking worth a refund and an angry guest. That split is the single most important lesson in this whole design.

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Booking/inventory/payments (source of truth)	PostgreSQL/MySQL (relational)	Multi-row ACID transactions (check + decrement 3 date rows + insert booking, atomically), strong consistency — inventory must never go negative. <i>Not a KV/document store</i> — you'd hand-roll cross-row locking/compensation, reinventing what a relational engine gives free.
Preventing oversell	Optimistic lock (version column) to win a HOLD	Wins the race without holding a lock across a slow human payment step; version mismatch = "retry or fail," cheap under normal contention. <i>Not a bare SELECT FOR UPDATE held across payment</i> — pins a lock on the hottest rows for as long as a human types a card number.
The reserve-then-pay gap	HOLD row + TTL, background sweeper	Firm window to pay without blocking other bookers; unpaid holds auto-expire and return inventory. <i>Not confirming immediately without a hold</i> — a checkout abandon then has no path to reclaim the room.
Availability search	Elasticsearch, fed async via CDC/outbox from Postgres	Filter/sort/paginate over city+dates+price+rating — exactly its job — tolerates seconds of staleness. <i>Not querying Postgres directly</i> — 1,000-2,000 QPS of ad-hoc reads would compete with, and could starve, the transactional path.
Hot-page / result caching	Redis, cache-aside, short TTL	Absorbs repeat searches for popular city/dates at memory speed. <i>Not caching the booking-decision number</i> — \$10, that must stay live.
Holds with auto-expiry	hold_expires_at column + sweeper (or Redis TTL key)	TTL-based expiry needs no manual cleanup; reclaims abandoned holds automatically.
Idempotency (booking + payment retries)	Client Idempotency-Key, stored in the same txn as the side effect	A flaky-network retry must return the <i>original</i> result, not re-execute — otherwise a timeout-retry double-books or double-charges.
Async fan-out (index refresh, email, analytics)	Kafka	Durable, partitioned, replayable — a slow email provider or index refresh can never block a confirmed booking. <i>Not synchronous calls to every downstream</i> — ties booking latency to the slowest consumer.
Static assets (photos, page shell)	CDN	Near-zero origin load for content that rarely changes; frees the fleet for actual search/booking traffic.
Edge routing	API Gateway (L7)	Path-based routing (/search → large read fleet, /bookings /pay → small guarded

Concern	Choice	Why this, and why not the alternative
		fleet), auth, rate limiting — all content-aware decisions a plain L4 balancer can't make.

Say-it-out-loud justification: "Postgres with row-level/optimistic locking is the ACID, strongly-consistent core that guarantees we never oversell a room-night; Elasticsearch plus Redis form a completely separate, horizontally-scaled read path that absorbs 1,000x the QPS of booking because search can tolerate staleness that booking never can; a short-lived HOLD decouples the fast 'win the race' decision from the slow human payment step; and Kafka moves every non-essential side effect off the booking critical path so a slow downstream can never turn into a failed or delayed confirmation."

12. Consistency model — what's strong, what's eventual

- **Strong / linearizable:** the `RoomInventory.booked_rooms` decision for a given (hotel, room type, date) — enforced by the version-check/lock inside one Postgres transaction, always against the primary, never a replica or cache.
- **Read-your-own-writes:** the guest who just booked sees `HELD` / `CONFIRMED` immediately.
- **Eventual:** the Elasticsearch index, the "X rooms left" badge, cross-region search copies, confirmation emails, analytics — all refresh within seconds via CDC/Kafka, and that lag is explicitly acceptable (§10).
- **The one rule that makes replica/index lag safe:** the booking transaction **always re-validates against the primary**, regardless of what a stale search result showed the guest a moment earlier. Skip that re-validation "because search already said available," and replica/index lag turns directly into oversells.

13. Failure modes & graceful degradation

Failure	What happens	Mitigation
Payment succeeds, booking-confirm write fails mid-way	Guest charged but booking not <code>CONFIRMED</code>	Write <code>HELD</code> /payment <code>PENDING</code> in one local txn <i>before</i> calling the gateway; reconciliation compares gateway vs. booking status and fixes it, or refunds
Idempotency key reused (client retry)	Would double-book/double-charge if unguarded	Return the stored result of the original attempt; never re-execute the side effect
Hold expires before payment completes	Room stuck <code>HELD</code> , "sold out" with nobody holding it	Sweeper flips expired <code>HELD</code> → <code>EXPIRED</code> , decrements <code>booked_rooms</code> , returns to pool
Elasticsearch down	Search degraded	Fall back to direct-replica query or "results may be incomplete"; booking unaffected — never depended on the index
Payment gateway down	Guest can't complete checkout	Fail fast, keep the <code>HOLD</code> alive to its TTL, let them retry; never auto-confirm without real payment success

Failure	What happens	Mitigation
DB primary down (hotel's home region)	Writes pause for that shard/region	Failover to a promoted replica for writes; reads keep serving from replicas/index throughout
Cross-region latency spike	Booking a far-away hotel feels slow	Acceptable — booking is rare (2.3/s avg); never trade data-locality/correctness for speed on a low-volume path

14. Follow-up curveballs

Interviewer: Why not just use `SELECT ... FOR UPDATE` for the whole flow, including while the user is on the payment page, and skip the HOLD entirely?

Candidate: Because that pins a row lock on the most contended inventory rows for however long a human takes to enter card details — seconds to minutes — collapsing throughput exactly where popularity is highest. The HOLD separates the **fast** decision (win the race, commit in milliseconds) from the **slow** external step (payment), so the lock/version-check only ever guards the instant of the actual state transition.

Interviewer: Two holds expire at nearly the same moment for the same room, and both users hit "pay" within the same second, right at the boundary. What happens?

Candidate: The `pay` handler **re-validates the hold before charging** — status and `hold_expires_at` inside a transaction against the primary. If the TTL lapsed, the sweeper (or the handler itself) has already flipped it to `EXPIRED` and released the room-night; at most one of the two `pay` calls can see a still-valid `HELD` row. We never charge a card without a currently-valid hold confirmed *at charge time* — that ordering is the invariant.

Interviewer: How would flights differ from hotels here?

Candidate: Same core problem — seat inventory per flight leg is a "room-night" equivalent ("seat-leg"); multi-leg itineraries need every leg checked and decremented atomically, like every night of a stay. The wrinkles: **fare classes/dynamic pricing** layered on the same row, and itineraries spanning *different airlines'* systems (codeshares), which pushes toward a saga/compensation pattern instead of one local transaction. The oversell-prevention mechanics carry over unchanged.

Interviewer: How do you keep the search index from drifting too far during a huge event, like a citywide festival selling out every hotel?

Candidate: Treat CDC/Kafka consumer lag as a first-class metric — a climbing lag is the earliest signal of drift; I'd rather show a "limited results" banner than let staleness grow silently. Since booking never trusts the index anyway, the worst outcome of drift is a slightly worse UX, never a correctness failure.

15. Deep-dive: why not just query the primary database for search?

Interviewer: You reached for Elasticsearch pretty fast. Why not skip it — search is just a `SELECT` with some `WHERE` clauses. Query the primary Postgres directly, cache the hot pages, and call it done. One less system to run and no staleness at all.

Candidate: It's the right instinct to *start* with — don't add a system you don't need — but this one specific access pattern breaks it, and it's worth being precise about why, because "just query the DB" is exactly the kind of thing that works in the demo and falls over in week two of production.

The read-amplification and the funnel

The whole shape of this problem is a **funnel**: from \$2, roughly **1,000-2,000 searches/sec at peak feeding only ~50-100 bookings/sec** — a 50:1 (or worse, once you count re-searches, pagination, and filter changes per session) browse-to-book ratio. That's already 20-40x more QPS than booking. Now amplify *within* one search:

```
ONE logical search request:
    "hotels in Paris, Aug 12-15, 2 guests, sort by price, filter: 4-star, pool, breakfast"

...is NOT one cheap row lookup. To answer it against the relational schema (§5) you must:
- find every Hotel in/near Paris                (geo filter)
- join to every RoomType at each of those hotels (fan-out #1)
- join to RoomInventory for EACH of the 3 requested nights, per room type (fan-out #2)
- compute available = total - booked for all of them, then filter "still available"
- re-filter by star rating, amenities (unindexed facets), then SORT by price, PAGINATE

500 candidate hotels x 8 room types x 3 nights ~= 12,000 row reads for ONE search request.
At 2,000 searches/sec that's ~24,000,000 row reads/sec landing on the SAME primary
that booking needs for its transactions.
```

That's the **read amplification**: one user-facing search "read" fans out into thousands of physical row reads and joins — and it's happening at 20-1,000x the rate of the writes the primary actually exists to protect.

Why geo + date-range + facets specifically wrecks an OLTP schema

A relational primary is tuned for **point lookups and short transactions on a known key** (§5's composite PK (`hotel_id`, `room_type_id`, `date`)) — exactly what booking needs. Search needs the opposite: **ad-hoc, multi-predicate, ranked queries** —

- **Geo** ("hotels within 5km of this point") wants a spatial index (R-tree/geohash) walked *together with* —
- **Date-range availability** (a join/aggregation across N nights per candidate), *together with* —
- **Facets** (star rating, price band, amenities, free-text hotel name) that don't share a single useful composite B-tree index — every new filter combination is either a fresh index you didn't build or a fallback to scanning too many rows.

No single B-tree-oriented index serves all three well at once; you'd end up either building and maintaining a pile of composite indexes that still don't cover every filter combination, or falling back to sequential scans under load. Elasticsearch/OpenSearch exist precisely for this: inverted indexes for facets/text, native geo-point/geo-distance queries, and aggregations for "X rooms left" counts and filter facets, all denormalized ahead of time so a query at request time is cheap.

The tradeoff you're accepting: staleness

The reason this isn't a free win is that a dedicated index is a **copy**, and copies drift. Query-the-primary gives you zero staleness by construction (you're reading the live rows); a separate index means a booking that just confirmed might not show as "sold out" in search for a few seconds. §4 and §10 already established why that's an acceptable trade for *this* problem — the booking decision never trusts the index anyway — but it is a real cost: extra infrastructure, an async pipeline that can lag or break, and one more thing to monitor.

Concern	Query the primary DB at request time	Dedicated search index (Elasticsearch)
Throughput isolation	None — search QPS and booking txns compete for the same CPU/IO/connections	Full — search load never touches the primary at all
Query fit (geo+date+facets)	Poor — no single index serves spatial + range + facet + sort together; combinations force scans	Purpose-built — geo-point queries, inverted indexes for facets/text, aggregations, all native
Latency under load	Degrades sharply as concurrent searches rise (lock contention, buffer-pool thrash)	Stays flat — horizontally sharded/replicated independent of booking load
Staleness	None — always the live number	Seconds of lag — availability may show slightly stale
Failure blast radius	A search spike can starve/slow booking transactions	Search and booking fail independently; ES outage doesn't touch booking (§13)
Operational cost	One system, simpler ops	Two systems + an async pipeline to build, monitor, and keep in sync
Scaling knob	Vertical scale / read replicas on the primary — expensive, still shares write capacity story	Add shards/replicas to the index — cheap, unlimited relative to booking

Keeping the index fresh without touching the booking path

The index is fed **asynchronously**, never synchronously from the booking transaction, so freshness never becomes a dependency booking has to wait on:

```

Postgres primary (RoomInventory/Bookings commit)
  | WAL / transaction log
  ▼
CDC connector (e.g. Debezium) or an outbox table written in the SAME txn as the booking
  | emits change events, at-least-once, ordered per row key
  ▼
[ Kafka: inventory-changes ] — durable, replayable
  |
  ▼
[ ES indexer worker ] → upserts the affected hotel/room-type/date documents

```

Using CDC (reading the WAL) or a transactional outbox — rather than "call Elasticsearch from inside the booking transaction" — means a slow or down indexer can **never** block or fail a booking; worst case, search is stale by however long the consumer lags, which is exactly the staleness the design already accepts (§9, §13).

Rule of thumb: *If a read needs to serve as the input to a correctness-critical write decision, it must hit the primary, in a transaction, every time — no exceptions. If a read only needs to be "close enough" and is dominated by ad-hoc multi-predicate ranking/filtering at far higher QPS than the writes, split it into its own purpose-built, asynchronously-fed index. Mixing the two onto one system optimizes neither.*

16. Deep-dive: the overlapping-date-range double-book, pessimistic vs. optimistic

Interviewer: Let's go one level harder than §6. Same room type, **last room** — but this time Guest A requests nights **Aug 12-15** and Guest B requests **Aug 14-17**. Their ranges only *overlap* on the night of the 14th, and both hit "book" in the same instant. Walk me through exactly how this oversells, then show me both a pessimistic and an optimistic fix — and tell me which you'd actually ship.

Candidate: Good, because this is harder than the single-night case in §6: the two transactions don't touch identical rows, they touch **overlapping sets** of rows — Guest A locks/reads nights {12,13,14}, Guest B locks/reads nights {14,15,16}. The only room-night that can't be sold twice is the 14th. That partial overlap is exactly what makes naive read-check-write dangerous and makes lock *ordering* matter for the pessimistic fix.

How the naive read-check-write oversells

Assume `RoomInventory(hotel, room_type, date)` has exactly **1 room left on the 14th**, and plenty of rooms on the other nights.

time →

Guest A (nights 12,13,14)

Guest B (nights 14,15,16)

SELECT available(12) = 5

SELECT available(13) = 3

SELECT available(14) = 1 ← both read "1" before either writes

SELECT available(14) = 1

SELECT available(15) = 4

SELECT available(16) = 6

all nights > 0 → proceed

UPDATE booked(12..14) += 1

all nights > 0 → proceed

UPDATE booked(14..16) += 1

Result: booked_rooms(14) went from (total-1) to (total-1)+2 = total+1.

BOTH bookings report CONFIRMED. The 14th is oversold by one room.

The bug is identical in shape to §6's single-night race — read-then-write with no serialization between the two — but it's easy to miss in review because the two bookings "don't look like" they're fighting over the same thing: their date ranges aren't equal, only their **intersection** is.

Fix 1 — pessimistic: SELECT ... FOR UPDATE on every touched row, in a fixed order

Lock every room-night row the *specific* booking touches, inside one transaction, before checking availability:

Guest A txn

BEGIN;

SELECT * FROM room_inventory

WHERE (hotel,room_type,date)

IN ((h,rt,12),(h,rt,13),(h,rt,14))

ORDER BY date FOR UPDATE;

locks 12,13,14

▼

check available>0 for 12,13,14 → yes

UPDATE booked_rooms +=1 for 12,13,14

COMMIT; → releases locks

Guest B txn

BEGIN;

SELECT * FROM room_inventory

WHERE (hotel,room_type,date)

IN ((h,rt,14),(h,rt,15),(h,rt,16))

ORDER BY date FOR UPDATE;

tries to lock 14 → BLOCKS

(A already holds it)

↓

lock on 14 granted, re-reads:

available(14) = 0 now (A took it)

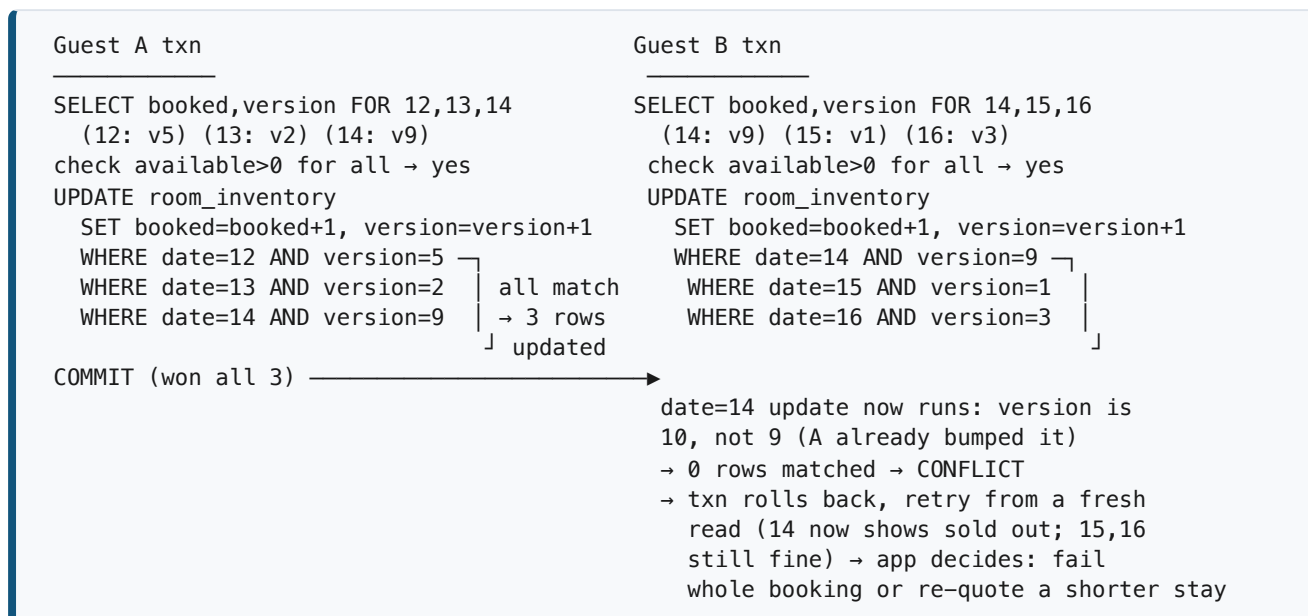
→ fails this txn, returns SOLD_OUT for 14

(Guest B can retry for 15-17 only, etc.)

Why ORDER BY date (or a canonical row order) matters here specifically: because the two ranges overlap only *partially*, if each transaction locked rows in a different order (A locks 14 before 12; B locks 16 before 14) they could each hold one row the other wants and **deadlock** — the classic overlapping-range hazard that a same-row race (§6) never exposes. Locking touched rows in a **consistent global order (e.g., sorted by date)** across every booking transaction avoids that deadlock entirely; the DB's deadlock detector is a fallback, not a substitute for this.

Fix 2 — optimistic: version/compare-and-set, retry on conflict

Read (booked_rooms, version) for every touched night with a plain SELECT (no lock), then attempt a conditional update per row:



No lock is ever held; contention is only discovered at `UPDATE` time via the version check, and the loser simply **retries against fresh data** (or gives up and reports `SOLD_OUT` for the conflicting night) instead of blocking behind Guest A.

Comparing them for this exact race

	Pessimistic (<code>SELECT FOR UPDATE</code>)	Optimistic (version / CAS)
Contention behavior	Guest B <i>blocks</i> , waits, then re-checks — deterministic, no wasted work retried	Guest B's txn runs fully, then is <i>thrown away</i> and retried — wasted work under high contention
Deadlock risk	Real, because ranges overlap partially — must lock in a consistent global order	None — no locks held, so nothing to deadlock on
Behavior under low contention	Slight overhead of acquiring locks nobody's fighting over	Near-zero overhead — the common case (no conflict) is a single cheap <code>UPDATE ... WHERE version=x</code>
Behavior under high/hot contention	Degrades gracefully — a queue of waiters, each resolved in turn	Degrades badly — many retries can pile up and re-collide (thundering retry) without backoff/jitter
Latency shape	Waiters pay queueing delay	Winners are fast; losers pay a retry round-trip, but never block others
Best when	A hotel is genuinely viral / very hot (few rooms, many simultaneous contenders on the <i>same</i> rows)	The overwhelming common case — most rooms/nights, most of the time, see little to no contention

The recommended default

Optimistic, by default — same conclusion as §6, but now with the caveat this scenario adds: for the rare very hot room-night (a viral hotel, a sold-out festival weekend), optimistic retries can start colliding with each other under extreme contention, at which point falling back to pessimistic locking (with the consistent lock-ordering rule above, to stay deadlock-free) for just that hot inventory is the right

escalation — not the default for every booking. Mirroring §6's HOLD design: whichever mechanism wins the race only guards the short "commit the HOLD" instant; the slow human/payment step never sits behind either kind of lock.

What to say out loud: *"A naive read-check-write oversells the one night where two overlapping ranges intersect, because both transactions read 'available' before either writes. Pessimistic locking fixes it by locking every touched room-night row up front, in a consistent order across all bookings so overlapping-but-not-identical ranges can't deadlock each other; optimistic locking fixes it with a version compare-and-set that holds no lock and just retries losers. I'd default to optimistic since most nights aren't contended, and reserve pessimistic locking for the rare, genuinely hot room-nights where retries would otherwise collide with each other."*

Summary — the mental model

This problem is really **two systems wearing one UI**: a **search funnel that's a pure scale problem** — solved with read replicas, Elasticsearch, Redis caching, and a CDN, all tolerant of staleness — and a **booking path that's a pure correctness problem** — solved with a relational database, per-date "room-night" rows, optimistic/pessimistic locking to win the race atomically, and a short-lived TTL hold that separates the fast commit from the slow human payment step. The single rule that ties the two halves together safely: **the booking decision always re-reads the authoritative primary inside a transaction and never trusts a cache, a replica, or the search index** — which is exactly what makes staleness on the read side harmless. Async event fan-out (Kafka) keeps notifications, index refresh, and analytics off the booking critical path, and multi-region keeps each hotel's authoritative inventory anchored to one home-region primary while search is replicated everywhere for speed.